

Задание №2.

Простое MVVM-приложение

Цели задания:

- Создать Wpf-приложение с использованием разделения View и ViewModel.
- Познакомиться с концепцией привязки данных (Data Binding).

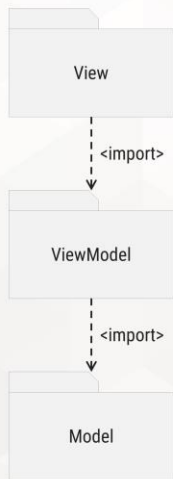
Паттерн MVVM

Ранее в приложениях мы придерживались концепции разделения программы на два слоя – Model (бизнес-логика) и View (пользовательский интерфейс), где View включает в себя использование Model, но Model не должен знать и обращаться к элементам View. Это фундаментальное разделение, которое позволяет обеспечить меньшую связность классов в программе, повышает возможность повторного использования классов, простоту поддержки, тестируемость. Косвенно подход позволяет обеспечить большую независимость от платформы, т.к. при переносе ПО на другую платформу достаточно будет написать новый пользовательский уровень (слой View) без изменений в слое бизнес-логики Model.

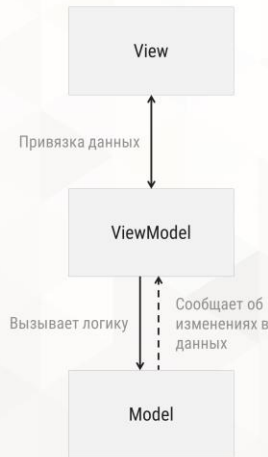
Развитием подхода разделения ПО на слои Model и View стали паттерны MVC, MVP, MVVM, VIPER и другие более сложные паттерны.

Паттерн MVVM – высокоуровневый, или архитектурный паттерн проектирования приложения, предполагающий разделение приложения на три слоя: Model («представление», логика предметной области), View («вид», пользовательский интерфейс) и ViewModel («представление вида», логика пользовательского интерфейса). Паттерн активно применяется в десктоп и мобильных приложениях.

Структура паттерна
на диаграмме пакетов



Структура паттерна
в виде взаимодействия
уровней



Содержит верстку пользовательского интерфейса, не содержит логику по обработке данных, и получает данные и вызывает логику из ViewModel

Посредник между View и Model, содержит логику обработки действий пользователя и передачи данных – во View. Не знает напрямую про View, передаёт данные во View. Уведомляет с помощью событий об изменении данных в Model

Содержит классы, описывающие предметную область. Не знает про сущности View и ViewModel

Уровни MVVM:

Model – как и в других MV*-паттернах, уровень описывает предметную область: её структуры данных, проверку правильности данных, процесс обработки данных для решения задач пользователей. В структуре является самым низким уровнем, который ни в коем случае не должен что-то знать про пользовательский интерфейс в лице View и ViewModel. Уровень ViewModel хранит объекты классов Model, вызывает их методы, но классы Model не обращаются к ViewModel. Model может уведомлять об изменениях данных с помощью событий или, в случае ошибок, генерировать исключения для обработки на верхних уровнях архитектуры.

View – верстка и поведение пользовательского интерфейса, которые видит конечный пользователь и с которыми он взаимодействует. Отображает пользователю данные из уровня Model, позволяет работать с ними (создавать, просматривать, редактировать, удалять). Важно, что сам View не содержит логику действий пользователя по обработке данных – это делегируется уровню ViewModel. Сам же уровень View может содержать логику по поведению элементов пользовательского интерфейса, например, изменение цветов, стилей, анимации в зависимости от действий пользователя или результатов обработки данных на уровне Model или ViewModel.

ViewModel – фактически, уровень является абстракцией пользовательского интерфейса, предоставляющий для View данные для отображения в интерфейсе и команды, которые должны выполняться при действиях пользователя. Уровень является связующим звеном между View и Model, передающий данные между ними и обрабатывающий события. При этом ViewModel (далее VM) не должен знать ничего о конкретных объектах или сущностях уровня View.

При запуске программы, некий менеджер или контроллер (в случае языка C# это классы Program или App) создаёт экземпляры классов из View и VM. Каждому объекту View соответствует объект уровня VM. Объект View агрегирует объект VM. При реализации View, в верстке указываются ссылки элементов управления с открытыми свойствами или методами VM – например, текстовое поле хранит ссылку на вещественное свойство из VM, а кнопка содержит ссылку на открытый метод VM. С помощью механизма привязки данных (Data Binding) данные из вещественного свойства будут отображаться в текстовом поле, а при изменении данных в текстовом поле автоматически обновится значение в вещественном свойстве. Аналогично, по нажатию на кнопку, с помощью привязки данных вызовется соответствующий метод VM. Таким образом, VM ничего не знает о текстовых полях, выпадающих списках, флажках или кнопках в пользовательском интерфейсе, но содержит логику действий при работе пользователя с этими элементами. Сам уровень VM в свою очередь агрегирует объекты Model, вызывает их методы, а результаты вызова методов отдаёт на отображение пользователю во View с помощью Data Binding. Иногда на схемах механизм Data Binding представляют четвертым уровнем данного паттерна, или заменяют уровень VM на уровень Binder (паттерн в таком случае называют Model-View-Binder). Суть паттерна и разделения обязанностей по уровням от этого не меняется.

Паттерн MVVM был придуман для того, чтобы искусственно выделить код пользовательского интерфейса (так называемый “code-behind”) из уровня View в отдельный уровень с помощью DataBinding. Примером code-behind служат обработчики событий в Windows Forms. Элементы управления предоставляют события, на которые подписываются обработчики внутри самих форм. При этом обработчики, находясь в одном классе с элементами управления, могут напрямую обращаться к ним, обновляя на них данные. В итоге связь обработки действий пользователя с самими элементами управления очень высокая. В паттерне MVVM связь направлена в другую сторону – это элементы управления хранят ссылку на методы в VM, а уровень VM ничего не знает о конкретных элементах управления или окнах. Такой подход даёт несколько важных преимуществ по сравнению с обычным разделением Model-View:

- 1) Уровень View содержит только верстку пользовательского интерфейса, без какой-либо логики. Таким образом, UX- и UI-дизайнеры могут верстать пользовательский интерфейс на уровне View, не касаясь исходного кода, который теперь находится на уровне ViewModel. Работа дизайнеров и программистов разделена. В отличие от Windows Forms, где верстка и обработчики событий находятся в одной сущности, что требует от дизайнера знаний языка программирования, а одновременная работа дизайнера и программиста над одним окном или элементом управления затруднительна.
- 2) VM становится еще одним значительным фрагментом кода, который не зависит от платформы. Благодаря этому, разработка приложения под разные платформы упрощается, почему паттерн и используется в мобильной разработке. Если вам необходимо разработать приложение, работающее под iOS и Android, вам потребуется разработать только две отдельные верстки View для каждой платформы, а уровни Model и VM будут общими. В случае обычного разделения Model-View, вам потребуется переписать для каждой платформы не только верстку, но и весь code-behind, что может в несколько раз увеличить объем работ.
- 3) Разделение обязанностей на большее количество низкосвязанных уровней помогает разработчикам лучше понимать структуру программы, а, следовательно, писать более качественный код.

Реже отмечают преимущество MVVM с точки зрения тестирования. Являясь классами, не использующими элементы пользовательского интерфейса, классы уровня VM могут быть покрыты автоматизированными интеграционными или юнит-тестами. Таким образом, тестирование логики окон можно делать не вручную, а автоматизировано.

К минусам паттерна MVVM можно отнести его применимость к приложениям средней сложности. Если приложение простое и состоит из пары пользовательских окон или страниц, применение паттерна MVVM может быть избыточным по сравнению с реализацией приложения обычным MV-разделением с обширным code-behind, как это реализуется в Windows Forms с помощью обработчиков событий. Для крупных приложений, состоящих из сотен окон и страниц пользовательского интерфейса, перед разработчиком встают проблемы грамотного масштабирования архитектуры MVVM, в противном случае это приведет к падению эффективности (производительности) приложения. Впрочем, правильное масштабирование архитектуры остается открытым вопросом для любого архитектурного паттерна.

Паттерн MVVM лег в основу WPF, а затем Xamarin и MAUI. WPF - это технология от Microsoft для создания десктоп-приложений, пришедшая на смену Windows Forms. Технологии Xamarin и MAUI появились как технологии для кроссплатформенной разработки, в частности, для создания мобильных приложений. Все три технологии очень похожи друг на друга, так как основаны на платформе .NET и применении паттерна MVVM.

Для упрощения разработки WPF-приложений, существуют сторонние библиотеки и фреймворки, предоставляющие готовые элементы для реализации MVVM-паттерна:

- **Galasoft MvvmToolkit Light** – не поддерживаемый, но широко используемый фреймворк; получил популярность благодаря своей простоте; могут быть проблемы при использовании с .NET 6 и выше.
- **.NET Community Toolkit MVVM** (или MVVM Toolkit) – новый фреймворк, пришедший на замену Galasoft MvvmToolkit Light; архитектурно его повторяет, является решением от Microsoft; относительно новый, поддерживается платформой .NET Standard 2.0, .NET 6 и выше. Для более ранних версий платформы придется использовать решение от Galasoft.
- **Prism Library** – более функциональная и тяжеловесная библиотека.
- **Caliburn, DevExpress MVVM, Uno** и др.

В рамках данного курса заданий мы изучим разработку MVVM-приложений с использованием .NET Community Toolkit MVVM. Однако первое приложение для ознакомления будет сделано без использования сторонних паттернов.

Перед выполнением задания необходимо ознакомиться с источниками [1, 2].

Задание

1. Создайте в репозитории Programming ветку:

`features/2_simple_mvvm_application`

В течение работы не забывайте делать коммиты в ветку, т.к. многие элементы могут не получаться с первого раза.

2. Создайте в репозитории Programming новое решение Contacts:

- проект (csproj) должен называться View;
- тип проекта WPF Application;
- решение (sln) должно называться Contacts;
- папка решения должна располагаться в папке src.

3. Разработайте простое ознакомительное MVVM-приложение. В первую очередь, создайте в проекте подпапки Model и ViewModel. Позже данные папки будут вынесены в отдельные проекты, а пока в этих подпапках будут храниться классы, соответствующие этим уровням.

4. Добавьте в папку Model новый класс Contact, хранящий имя, номер телефона и почту контакта. Добавьте свойства и конструкторы.

5. Сверстайте главное окно MainWindow согласно макету:

The screenshot shows a WPF window titled "Contacts" with a standard Windows title bar (minimize, maximize, close buttons). The window is divided into two main vertical panels. The left panel is titled "Edit Contact" and contains three text input fields: "Name:" with the value "Смирнов Юрий", "Phone Number:" with the value "+7-913-111-22-33", and "Email:" with the value "yuri.smirnov@no.mail". The right panel is titled "Read Contact" and displays the same information in a read-only format: "Name:" followed by "Смирнов Юрий", "Phone Number:" followed by "+7-913-111-22-33", and "Email:" followed by "yuri.smirnov@no.mail". At the bottom right of the window, there are two buttons: "Save" and "Load".

Поля окна – 15 пкс с каждой стороны;

Отступы всех элементов управления, кроме StackPanel или Grid – 3 пкс;

Размеры кнопок – 75x25 пкс.

6. Добавьте в папку ViewModel класс MainVM (VM для главного окна). Класс должен:

- реализовывать стандартный интерфейс INotifyPropertyChanged;

- предоставлять свойства Name, PhoneNumber и Email для привязки со стороны MainView;
- экземпляр контакта Contact, в котором должна храниться вся актуальная информация с пользовательского интерфейса.

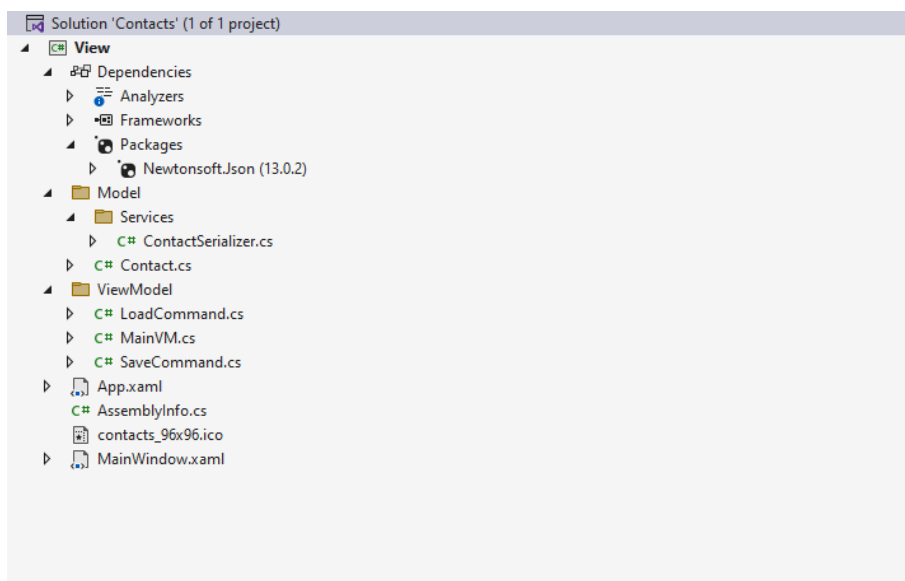
7. Добавьте в папку Model подпапку Services, и в неё добавьте новый класс ContactSerializer. В текущей версии программы класс должен предоставлять два метода на сохранение и загрузку одного объекта контакта из файла. Путь до файла хранится внутри класса в виде свойства, путь по умолчанию – «Мои документы\Contacts\contacts.json». Для реализации сериализации подключите библиотеку Newtonsoft.Json в качестве Nuget-пакета.

8. Добавьте в папку ViewModel два класса: SaveCommand и LoadCommand. Оба класса должны реализовывать стандартный интерфейс ICommand, где в методе Execute() выполняется сохранение или загрузка контакта из файла. Метод CanExecute() всегда возвращает true.

9. В конструкторе класса MainWindow сделайте инициализацию свойства DataContext новым объектом MainVM – это свяжет главное окно с конкретным объектом VM. Без инициализации DataContext привязка данных в главном окне работать не будет.

10. В верстку MainWindow добавьте привязку (binding) главного окна к элементам MainVM. Важно, что при вводе данных в поля слева, текстовые поля на правой панели окна должны обновляться сразу же, по нажатию каждой клавиши клавиатуры, а не при потере фокуса с текстового поля (см. Binding UpdateSourceTrigger).

11. Убедитесь, что программа работает корректно в соответствии с заданием. Проверьте правильность верстки и оформление кода, наличие всех комментариев. Структура программы должна быть следующая:



12. Одной из главных задач, которую вы должны решить в ходе выполнения работы, это определить, как в команду SaveCommand передать сохраняемый объект Contact, а из команды LoadCommand забрать загруженный из файла объект контакта. Решений может быть несколько, каждое со своими плюсами и минусами. Однако абсолютно неправильным решением будет использование на уровне VM любого обращения к элементам пользовательского интерфейса – это прямое нарушение паттерна MVVM.

13. Также не запрещается реализовать команды с помощью сторонних MVVM-библиотек, таких как .NET Community Toolkit MVVM.

14. Если задание выполнено правильно, то классы уровня Model и ViewModel

Литература

1. **Мак-Дональд М.**
WPF 4: Windows Presentation Foundation в .NET 4.0 с примерами на C# 2010 для профессионалов пер. с англ. – М.: ООО «И.Д. Вильямс», 2011. – 1024 с. : ил. – парал. тит. англ. (главы 1-9)
2. **Metanit.com. Руководство по WPF**
 / Metanit.com. – URL: <https://metanit.com/sharp/wpf/> (главы 1-7, 10)